

Exercise 2 – Processing of music data

Introduction

In this exercise we will learn how to make a minimal data engineering project that retrieves, cleans, validates and gets insights from a data source. The main code for work will be given, and you will have to understand and modify it for the first part.

Context

In this exercise you will be provided with a working code that gets lyrics and chords data from <https://lacuerda.net>. This is a well-known website for those who want to learn any song with the guitar, as it is very straightforward.

The code can be found in this repo, that we will use for other class exercises: <https://github.com/victorbrub/data-engineering-class>. You can either clone it in your local machine or download directly from GitHub.

Please, take time to review the website. You will see that:

- You can search for the artists you want.
- Each artist has a page with all its songs available.
- The song can have various versions. The code only gets the first one.
- The songs are uploaded by the community so there is not guaranteed quality, and comments are added by the users.

This is an example of a song uploaded to the website:

```
He Corregido la Posicion de los Acordes, de la
transcripcion de de Rubén Sánchez

(Rasgueo de acordes con los dedos)

Am
Y el del medio de los Chichos

G
Se me ha aparecido en sueños

F
Se me ha aparecido y me ha dicho

E
De tu rumba soy el dueño
Vamos Jose...

Am
Y el del medio de los Chichos

G
me ha dicho que en la otra vida

F
se le han olvidao las penas

E
se acuerda de la alegría.
```

1. Estopa - El de en medio de los chichos

Scrapping the data

Important:

You don't need to change anything about the scrapping logic in this exercise.

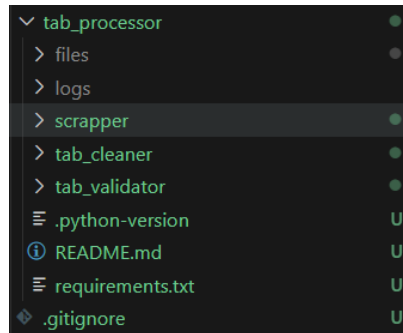
The code does what is known as *web scrapping*, that is: making a request to the website and downloading the html file. You can check that a call like `request.get(https://lacuerda.net)` from the **requests** python package gives you the static html code. There are though some python libraries that makes things easier, like [BeautifulSoup](#) , the one we are using in the code.

Before scrapping a website, you need to check for the legal conditions and permissions of the website. All websites have robots.txt file with that information. You can check it here <https://lacuerda.net/robots.txt>. Here the website is telling us that **All bots should wait at least 5 seconds between each request to our site**. Yes, we are a bot. But as we are only executing this on demand it does not apply. These conditions give the website the authority to cut all traffic from any IP that does not meet the conditions.

Along with the information scrapped, we will use the library **musicbrainzngs** to complete the information about the artists, if available.

Code

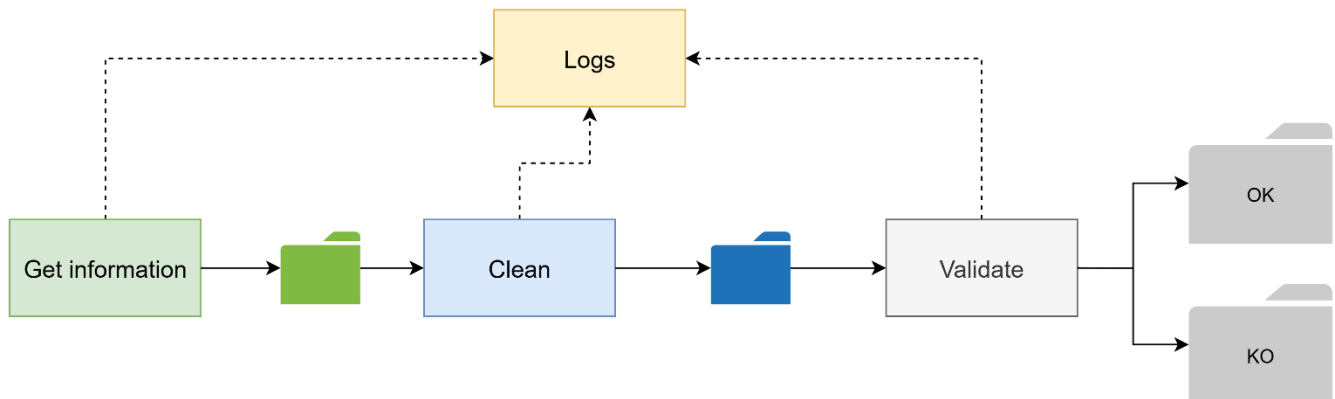
The code has the following structure:



There are three modules to check out:

- **scrapper**: gets the catalog and songs from lacuerda.net. This is where we download the data.
- **tab_cleaner**: gets the files downloaded by the scrapper and cleans it based on conditions defined.
- **tab_validator**: gets the files cleaned by the cleaner and validates the format based on conditions defined. It separates the output into OK files and KO files.

Also, the **files** directory is where all the files are stored, and the **logs** directory is where the execution logs are stored.



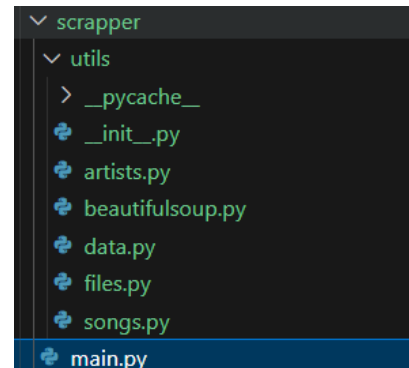
2. Flow diagram of the code.

The code works, but there are some issues that need to be addressed. Also, some functionality is needed to be this a proper project. Here you have the main parts of the code explained.

Scraper

The scraper logic is called from the 'main.py' file. A 'utils' directory has different functions modularized in different files.

- artists.py only retrieves the artist metadata from *musicbrainzngs*.
- beautifulsoup.py has the functions that call directly the *beautifulsoup* library.
- data.py has two classes defined: **Artist** and **Song**.
- files.py has functions two works with files.
- songs.py has the logic to retrieve the songs and make the catalogs.

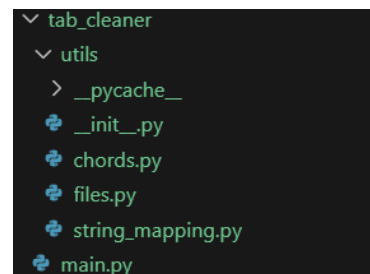


The output of the scraper are two kinds of files:

- Catalog files: .json files with artist catalog and full catalog.
- Song files: text files, one for each song scraped.

Cleaner

The cleaner, as its name says, cleans the files to remove comments, emails, dates or anything other than the lyrics and the chords. Or at least, try to.



Validator

The validator takes the cleaned files and checks if the format of the files is correct. If yes, it outputs the file to the OK folder, if not, to the KO folder.

Problem Statement

The problem is to solve the following issues and to add the functionality stated in this section.

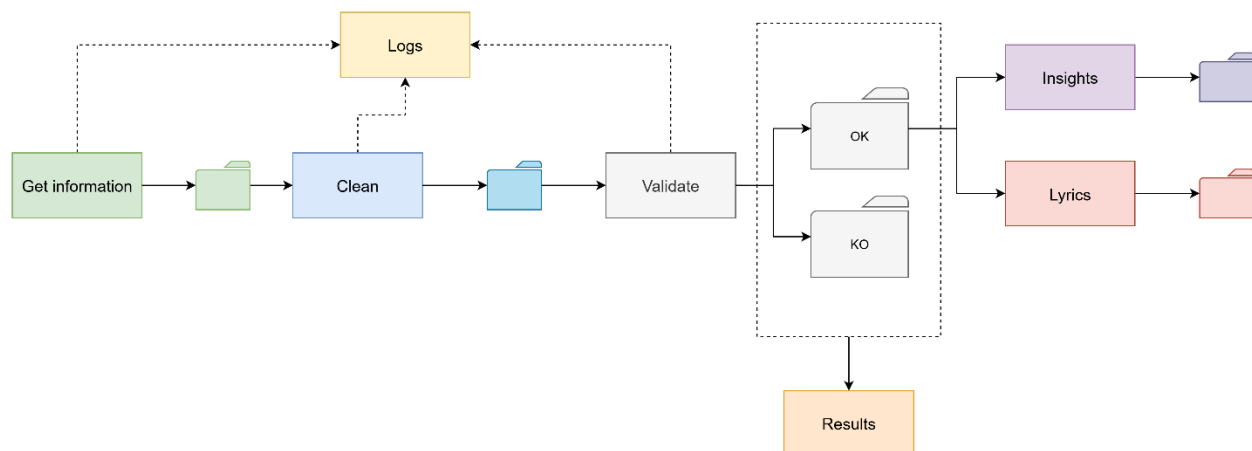
Issues to solve:

- (scrapper) Modify the **get_songs** in 'songs.py' file to use the catalog instead of scrapping again. **(2 points)**
- (scrapper) Check the logs for something strange. Try to know where those messages come from. No need to solve it, only get the origin. **(0.5 points)**.
- (cleaner) The cleaner is getting the catalogs also as inputs. Although it is not a problem for now, we need to avoid it. Implement a solution. **(0.5 points)**.
- (Validator) There is an issue when reading/saving the files as it is creating more directories than needed. **(0.5 points)**.
- (Validator) Implement an additional validation rule **(0.5 points)**
- Propose any change to make the code cleaner, clearer and better. **(0.5 points)**.

Functionalities to add:

- Add a new Python module called 'results' that checks the number of files we have for each output. **(0.5 points)**.
- Add a new python module called 'lyrics' that removes all the chords from the successfully validated files and stores it in the file's directory. **(2 points)**.
- Add a new python module called 'insights' that merges all OK lyrics into a single text file for each artist. Count, for each artist, the top 10 words (nouns, verbs, adjectives) that his lyrics have. Do it also globally, but with the top 20 **(2 points)**.
- Create a Python file that executes all modules in order. It has to have his own log file, and make sure that if any of the processes fail, is registered there **(1 point)**.

The final diagram should go something like this:



3. Diagram with added functionalities

In addition, you can have up to 2 extra points for adding more functionalities than the ones stated above.

Conditions

- The code must not have errors.
- All steps you follow must be properly documented the way you prefer.